

Day 123: API Request and Response Details

1. Day number

Day 123

2. Topic name

Headers, Path Parameters, Query Parameters, JSON, and Status Codes

Today you will look inside an API request and response and learn what additional information can travel between the client and server.

3. Connection

Yesterday, on **Day 122**, you learned how HTTP methods describe what a client wants to do:

```
GET    → Read
POST   → Create
PUT    → Update
DELETE → Delete
```

Today, you will go one level deeper.

An API request may contain more than just:

```
GET /products
```

It can also contain things such as:

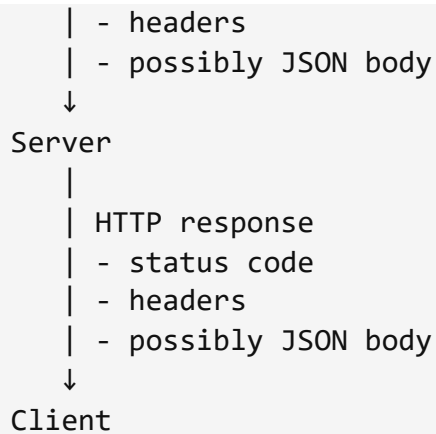
```
Headers
Path parameters
Query parameters
JSON body
```

And the server's response can contain:

```
Status code
Headers
JSON response body
```

The basic picture becomes:

```
Client
|
| HTTP request
| - method
| - URL
| - parameters
```



4. Important topics

Today you will focus on:

Headers
Path parameter
Query parameter
JSON request body
JSON response
HTTP status code

You do **not** need to memorize every possible HTTP detail.

The goal is simply to recognize the main parts of a typical API request and response.

5. Foundational notes

Think of an API request as a message containing several pieces of information.

For example:

```
POST /products

Content-Type: application/json

{
  "name": "Keyboard",
  "price": 1200
}
```

This request contains:

Method → POST
Path → /products
Header → Content-Type: application/json
Body → product data

The server may respond:

```
201 Created

{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

This response contains:

```
Status code → 201
JSON body   → created product
```

So HTTP communication is not just:

```
request → response
```

It is more like:

```
Request
├── method
├── URL
├── parameters
├── headers
└── body

Response
├── status code
├── headers
└── body
```

6. Headers

A **header** contains extra information about an HTTP request or response.

Think of headers as small pieces of **metadata**.

They describe the message rather than being the main data itself.

For example:

```
Content-Type: application/json
```

This tells the receiver:

```
| The body contains JSON data.
```

Imagine sending a parcel.

Parcel contents → actual data

Label on parcel → extra information about the parcel

Similarly:

JSON body → actual API data

HTTP header → information about the request/response

For today, you only need to understand the general idea.

Do not worry about learning many different HTTP headers yet.

7. Path parameter

A **path parameter** is a value that appears directly inside the URL path.

Example:

GET /products/25

Here:

25

can be treated as a path parameter representing a particular product.

You could imagine the API route as:

/products/{product_id}

Then:

/products/25

means:

product_id = 25

Another example:

/users/7

could mean:

```
user_id = 7
```

The path parameter normally helps identify a **specific resource**.

Simple mental model:

```
/products/25
      ↑
    path parameter
```

8. Query parameter

A **query parameter** is extra information added after a ? in the URL.

Example:

```
GET /products?category=electronics
```

Here:

```
category=electronics
```

is a query parameter.

Another example:

```
GET /products?limit=10
```

Here:

```
limit=10
```

is a query parameter.

It might mean:

```
| Return at most 10 products.
```

Query parameters are commonly useful for things like:

```
filtering
searching
sorting
limiting results
```

You only need the basic idea today.

9. Path parameter vs query parameter

This distinction is important.

Path parameter

A path parameter is part of the URL path.

Example:

```
/products/25
```

Here:

```
25
```

identifies a particular product.

Think:

```
Which exact resource?
```

Query parameter

A query parameter appears after ?.

Example:

```
/products?category=electronics
```

Here:

```
category=electronics
```

modifies or filters the request.

Think:

```
How should I search/filter the resources?
```

Easy comparison

Type	Example	Beginner meaning
Path parameter	/products/25	Give me product 25
Query parameter	/products? category=electronics	Give me products matching electronics

A simple memory trick:

Path parameter



Which resource?

Query parameter



Which options or filters?

For example:

```
/products/25
```

means:

```
specific product → 25
```

while:

```
/products?category=books
```

means:

```
products filtered by category → books
```

10. JSON request body

Sometimes the client needs to send structured data to the server.

For example, to create a product:

```
POST /products
```

The client could send:

```
{  
  "name": "Mouse",  
  "price": 500  
}
```

This is the **JSON request body**.

It contains the actual data being sent to the server.

Conceptually:

```
Client  
  |  
  | POST /products
```

```
|
| JSON body:
| {
|   "name": "Mouse",
|   "price": 500
| }
↓
Server
```

Remember that JSON often uses:

```
key : value
```

pairs.

For example:

```
{
  "name": "Mouse",
  "price": 500
}
```

contains:

```
name → Mouse
price → 500
```

11. JSON response

The server can also send JSON back to the client.

For example:

```
{
  "id": 25,
  "name": "Mouse",
  "price": 500
}
```

This is a **JSON response body**.

So JSON can travel in both directions.

```
Client
|
| JSON request
↓
Server
|
```

| JSON response
↓
Client

For example:

POST /products

Request body:

```
{  
  "name": "Mouse",  
  "price": 500  
}
```

Response body:

```
{  
  "id": 25,  
  "name": "Mouse",  
  "price": 500  
}
```

The server might have generated the new ID:

25

12. HTTP status code

An **HTTP status code** is a number in the server's response that tells the client what happened.

For example:

200

generally means the request succeeded.

Another example:

404

means the requested resource was not found.

Think of a status code as a short result message:

Request
↓

Server processes it



Status code tells client what happened

You only need six common codes today.

13. Status code 200 — OK

200 OK

means that the request succeeded.

Example:

GET /products/25

If product 25 exists, the server might return:

200 OK

with:

```
{
  "id": 25,
  "name": "Mouse",
  "price": 500
}
```

Beginner meaning:

200 → Success

14. Status code 201 — Created

201 Created

usually means that a new resource was successfully created.

Example:

POST /products

with:

```
{
  "name": "Monitor",
```

```
"price": 8000
}
```

The server creates the product and might respond:

```
201 Created
```

with:

```
{
  "id": 26,
  "name": "Monitor",
  "price": 8000
}
```

Beginner meaning:

```
201 → New resource successfully created
```

15. Status code 400 — Bad Request

```
400 Bad Request
```

generally means something is wrong with the request sent by the client.

For example, imagine the API expects:

```
{
  "name": "Keyboard",
  "price": 1200
}
```

but receives invalid data.

Conceptually:

```
Client
  |
  | incorrect request
  ↓
Server
  |
  | cannot correctly process it
  ↓
400
```

Beginner meaning:

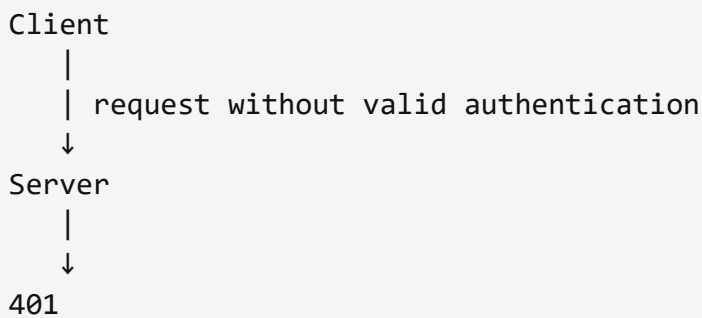
400 → Client sent an invalid request

16. Status code 401 — Unauthorized

401 Unauthorized

generally means authentication is required or the provided authentication information is not valid.

Imagine an API resource that requires the client to identify itself properly.



Beginner meaning:

401 → Authentication problem

Do not worry about login tokens or detailed authentication systems yet.

You will only remember the basic meaning today.

17. Status code 404 — Not Found

404 Not Found

means that the requested resource could not be found.

Example:

GET /products/999

Suppose product 999 does not exist.

The server might respond:

404 Not Found

Conceptually:

```
Client
  |
  | GET /products/999
  ↓
Server
  |
  | Search for product 999
  |
  | Not found
  ↓
404
```

Beginner meaning:

404 → Requested resource not found

18. Status code 500 — Internal Server Error

500 Internal Server Error

means something went wrong inside the server while processing the request.

Example:

```
Client sends valid request
      ↓
Server begins processing
      ↓
Unexpected server problem
      ↓
500
```

The important distinction is:

400 → something wrong with the client's request

500 → something went wrong on the server

Beginner meaning:

500 → Server-side problem

19. Common status codes summary

For Day 123, remember:

Code	Name	Simple meaning
200	OK	Request succeeded
201	Created	New resource created
400	Bad Request	Invalid client request
401	Unauthorized	Authentication problem
404	Not Found	Resource not found
500	Internal Server Error	Server problem

A useful pattern is:

2xx → generally successful

4xx → generally client/request problem

5xx → generally server problem

You do not need to learn other codes yet.

20. Easy example

Imagine this request:

POST /products?notify=true

Content-Type: application/json

Body:

```
{
  "name": "Webcam",
  "price": 2500
}
```

Let's identify its parts.

Method

POST

The intention is to create something.

URL/path

/products

The resource is products.

Query parameter

```
notify=true
```

It appears after:

```
?
```

Header

```
Content-Type: application/json
```

This tells the server that the request body contains JSON.

JSON body

```
{
  "name": "Webcam",
  "price": 2500
}
```

This is the new product information.

If the product is successfully created, the server might respond:

```
201 Created
```

with:

```
{
  "id": 30,
  "name": "Webcam",
  "price": 2500
}
```

21. Another example with a path parameter

Suppose the client sends:

```
GET /products/25
```

We can identify:

```
Method      → GET
Resource    → products
```

Path parameter → 25
JSON body → none needed for this simple example

If product 25 exists:

Status → 200

Response:

```
{  
  "id": 25,  
  "name": "Mouse",  
  "price": 500  
}
```

If product 25 does not exist:

Status → 404

Notice how the same request method can produce different status codes depending on what happens.

22. Problem statement

Examine this API request:

```
POST /products?notify=true  
  
Content-Type: application/json
```

Request body:

```
{  
  "name": "Laptop Stand",  
  "price": 1500  
}
```

Assume the request is valid and the server successfully creates the product.

Identify:

1. HTTP method
2. URL/path
3. Query parameter
4. Header
5. JSON request body

6. Expected success status code
7. Possible JSON response

Do not write Python or FastAPI code.

Focus only on recognizing the different pieces of the HTTP request and response.

23. Concepts used

The exercise combines everything from today's lesson:

```
HTTP method
URL
Header
Path parameter
Query parameter
JSON request body
JSON response
HTTP status code
```

It also uses your Day 121 and Day 122 knowledge:

```
Client
  ↓
HTTP request
  ↓
Server
  ↓
HTTP response
  ↓
Client
```

24. Thought process

When examining an API request, go through it in a consistent order.

Step 1: Find the HTTP method

Look for:

```
GET
POST
PUT
DELETE
```

Example:

```
POST /products
```

means:

```
method = POST
```

Step 2: Find the URL or path

Example:

```
/products/25
```

This tells you which resource is being accessed.

Step 3: Look for path parameters

Ask:

Is a value embedded directly inside the path?

Example:

```
/products/25
```

Then:

```
25 → path parameter
```

Step 4: Look for ?

If you see:

```
?
```

the information after it may contain query parameters.

Example:

```
/products?category=electronics
```

Then:

```
category=electronics
```

is a query parameter.

Step 5: Look for headers

Example:

```
Content-Type: application/json
```

This is extra metadata about the HTTP message.

Step 6: Look for a JSON body

Example:

```
{  
  "name": "Mouse",  
  "price": 500  
}
```

Ask:

Is the client sending actual structured data to the server?

If yes, that may be the request body.

Step 7: Determine what happened

Look at the server's status code.

For example:

```
200 → successful request  
201 → successfully created something  
400 → bad request  
401 → authentication problem  
404 → resource not found  
500 → server problem
```

25. Beginner-friendly request structure

You can imagine a request using this simple template:

```
HTTP REQUEST  
|  
|— Method  
|   POST  
|  
|— URL  
|   /products?notify=true  
|  
|— Query parameter  
|   notify=true  
|
```

```
├── Headers
│   Content-Type: application/json
└── Body
    {
      "name": "Mouse",
      "price": 500
    }
```

The server might respond:

```
HTTP RESPONSE
├── Status code
│   201
├── Headers
│   ...
└── Body
    {
      "id": 25,
      "name": "Mouse",
      "price": 500
    }
```

At this stage, you should be able to look at a simple request and identify its major parts.

26. Easy edge case: missing parameter

Suppose an API expects a particular product ID.

For example:

```
/products/{product_id}
```

A valid request might be:

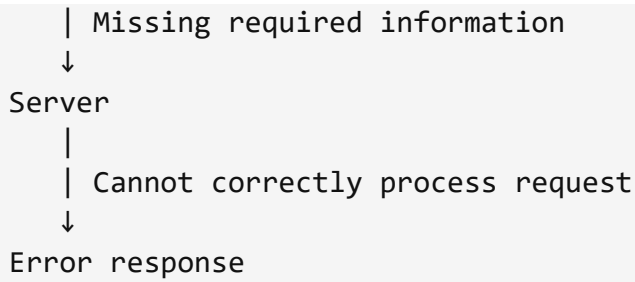
```
GET /products/25
```

But suppose the client sends something incomplete that does not provide the required product identifier.

The server may not be able to determine which product the client wants.

Conceptually:

```
Client
|
```



Depending on the API design, this may produce an error.

The key lesson is:

If the server requires information, the client must provide it in the expected place and format.

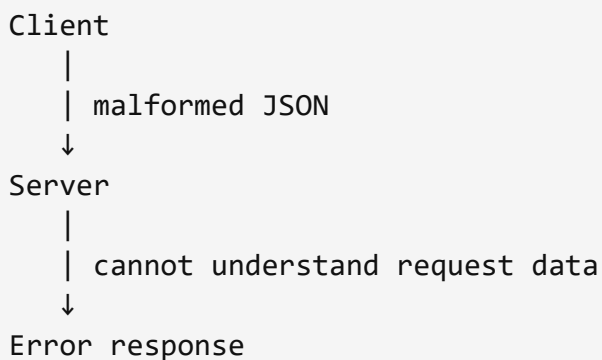
27. Easy edge case: malformed JSON

Correct JSON might look like:

```
{
  "name": "Mouse",
  "price": 500
}
```

Malformed data might be missing important JSON syntax.

Conceptually:



A request problem like this may result in a client-error response.

For your beginner mental model:

```
Malformed request data
  ↓
Could lead to a 400-type client error
```

You do not need to study JSON parsing errors in depth today.

28. Expected response

For the problem:

```
POST /products?notify=true
```

with:

```
{
  "name": "Laptop Stand",
  "price": 1500
}
```

you should eventually be able to explain the interaction like this:

The client sends a POST request.

The resource is /products.

The request contains the query parameter:
notify=true

The Content-Type header indicates JSON data.

The JSON request body contains the new product information.

If the server successfully creates the product,
a 201 status code is expected.

The server may return the newly created product
as a JSON response.

Conceptually:

```
Client
|
| POST /products?notify=true
|
| Content-Type: application/json
|
| {
|   "name": "Laptop Stand",
|   "price": 1500
| }
↓
Server
|
| Create product
↓
```

```
201 Created
  |
  | {
  |   "id": 31,
  |   "name": "Laptop Stand",
  |   "price": 1500
  | }
  ↓
Client
```

29. Hint only

For the exercise, inspect the request from top to bottom.

Start with:

POST

Ask:

What does POST represent?

Then inspect:

/products?notify=true

Look carefully at what comes after:

?

Next inspect:

Content-Type: application/json

Ask whether this is actual product data or information **about** the request.

Finally inspect:

```
{
  "name": "Laptop Stand",
  "price": 1500
}
```

Then think about which success code is normally associated with:

Create

Use only today's codes:

200
201
400
401
404
500

Day 123 key takeaway

Over the last three days, your API mental model has grown.

Day 121

Client → Request → Server
Client ← Response ← Server

Day 122

You added HTTP methods:

GET → Read
POST → Create
PUT → Update
DELETE → Delete

Day 123

Now you can inspect the contents of requests and responses:

```
Client
  |
  | REQUEST
  | Method
  | URL
  | Path/query parameters
  | Headers
  | JSON body
  ↓
Server
  |
  | RESPONSE
  | Status code
  | Headers
  | JSON body
  ↓
Client
```

The most important distinctions to remember are:

Path parameter → identifies something in the URL path
Query parameter → provides options/filtering after ?

JSON body → actual structured data being sent
Header → extra information about the HTTP message

200 → success
201 → created
400 → bad request
401 → authentication problem
404 → not found
500 → server problem

This gives you the foundation needed to understand what you are actually seeing when you begin sending requests to real APIs or building API endpoints later.

By the way, ChatGPT Images 2.5 can turn a rough idea into a finished image, with richer textures and details you can refine. Want me to create an image of a beginner-friendly infographic showing the anatomy of an API request and response?